



UNIVERSIDAD SERGIO ARBOLEDA

Big Data y Data Engineering

Parcial 1

Sistema de reservas de vuelos

Yslen Natalia Moreno Gutierrez
Oscar Mauricio Peñuela Guerrero

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 10 de septiembre de 2026

Índice

1. Contexto y alcance del documento	3
2. Supuestos de trabajo	3
3. Requisitos funcionales (5.1)	4
4. Requisitos no funcionales (5.2)	7
5. Modelo entidad-relación (5.3)	7
5.1. Diagrama	7
5.2. Decisiones de modelado y su justificación	8
5.3. Diccionario de datos	9
6. Arquitectura del backend (5.4)	12
7. Arquitectura del frontend (5.5)	13
8. Primera implementación del backend (5.6)	14
8.1. Alcance construido	14
8.2. Estructura del código	15
8.3. Mecanismo de concurrencia implementado	16
8.4. Evidencia de la prueba de concurrencia	16
8.5. Desviaciones y limitaciones declaradas del prototipo	17
9. Implementación en AWS: ETL, catálogo de datos y costos (5.7)	19
9.1. Arquitectura desplegada	19
9.2. Qué se copia al lado analítico y por qué	20
9.3. Registro en el Glue Data Catalog	21
9.4. Obstáculos técnicos resueltos durante la implementación	21
9.5. Justificación de los servicios AWS empleados	22
9.6. Proyección de costos	22
10. Trazabilidad cruzada	24
11. Limitaciones y trabajo futuro	24

1. Contexto y alcance del documento

Este documento recoge el diseño y la primera implementación funcional de un sistema de reservas para una aerolínea regional. Cubre, en orden, los supuestos de trabajo, los requisitos funcionales y no funcionales, el modelo entidad-relación, las decisiones de arquitectura de backend, la primera implementación del backend con su evidencia de pruebas, y la implementación en AWS del proceso ETL hacia una base de datos analítica registrada en el Glue Data Catalog.

El criterio que atraviesa todo el documento es la trazabilidad: cada decisión de arquitectura y cada servicio de infraestructura se justifica citando el requisito funcional (RF) o no funcional (RNF) que lo origina. La sección 10 consolida esa relación en una matriz.

El escenario de negocio entregado es deliberadamente incompleto. Los detalles no especificados se resuelven de forma explícita en la sección de supuestos: un supuesto no declarado es una decisión de arquitectura oculta, y por eso se documentan todos antes de modelar nada.

2. Supuestos de trabajo

Cada supuesto declara una decisión que el escenario dejó abierta, con su justificación. Los supuestos son la base de los requisitos: cuando un RF o una entidad del modelo existe por causa de un supuesto, se indica explícitamente.

ID	Supuesto	Justificación
S-01	Aerolínea regional con del orden de 20 rutas y una malla de vuelos recurrentes definida por horario semanal.	Fija el orden de magnitud sobre el que se calculan las métricas de rendimiento y el escenario de crecimiento de la proyección de costos.
S-02	Un itinerario con escalas es una sola reserva (un PNR) con N tramos asociados y ordenados, no varias reservas encadenadas.	El pasajero compra, paga, cambia y cancela el viaje como una unidad. Encadenar reservas obligaría a coordinar transacciones independientes y abriría la puerta a itinerarios parcialmente pagados.
S-03	Solo dos clases de cabina: ECONOMY y BUSINESS.	El escenario menciona ambas explícitamente y no sugiere más. Limitar el dominio permite modelar el inventario como un contador por clase.
S-04	El cambio de fecha está permitido y queda sujeto a disponibilidad. Si la nueva tarifa es mayor se cobra la diferencia; si es menor, no se reembolsa.	Regla de una sola vía, habitual en aerolíneas de bajo costo. Evita que el cambio de fecha se convierta en un mecanismo de reembolso encubierto.
S-05	Cancelación: si la reserva sigue en <i>hold</i> se libera sin costo; si está confirmada, el reembolso depende del tipo de tarifa (Básica no reembolsable, Flex reembolsable). En ambos casos el asiento se libera de inmediato.	Separa la política comercial (reembolso) de la operativa (inventario). El asiento debe volver a estar disponible sin esperar a que se resuelva el reembolso.
S-06	Existe programa de viajero frecuente con cuatro niveles (Básico, Plata, Oro, Platino) y acumulación de millas asociada a reservas confirmadas.	El escenario deja el punto abierto. Se asume su existencia porque condiciona el modelo de datos (entidades pasajero y transaccion_millas) y no podría añadirse después sin rehacerlo.

ID	Supuesto	Justificación
S-07	Un vuelo que se repite periódicamente se modela como vuelo_plantilla (número, ruta, horario, días de operación, vigencia) y cada ocurrencia en una fecha concreta como vuelo_instancia .	Sin esa separación, cada día de operación duplicaría la definición de la ruta y el horario, y no habría dónde anclar el inventario de una fecha específica.
S-08	El inventario se controla como un contador agregado por (instancia de vuelo, clase de cabina), no como filas de asiento individuales.	La garantía que exige el negocio es “no vender más asientos que los que tiene el avión”, no “reservar la silla 14C”. Un contador permite resolver la concurrencia con una única operación atómica sobre una fila.
S-09	La asignación de silla no forma parte de la reserva: ocurre en un proceso posterior de check-in.	Consecuencia directa de S-08. Se declara de forma explícita porque el enunciado pregunta específicamente por este punto.
S-10	Cada pasajero incluye una pieza de equipaje; las piezas adicionales se cobran y se asocian al pasajero dentro de la reserva.	El escenario deja el equipaje sin especificar. Asociarlo al par reserva-pasajero, y no a la reserva completa, permite que en un grupo cada pasajero lleve equipaje distinto.
S-11	El cobro se delega íntegramente a una pasarela de pago externa. El sistema registra el momento y el monto del pago, pero no almacena datos de tarjeta.	Reduce drásticamente el alcance de cumplimiento PCI DSS. Es la razón por la que el modelo no tiene una entidad <i>Pago</i> independiente.
S-12	La restricción específica del equipo está pendiente de asignación por parte del docente y se integrará de forma transversal cuando se conozca.	Se deja registrada como punto abierto para no presentar como cerrado un requisito cuyo contenido todavía se desconoce.
S-13	Se permite reservar sin pagar de inmediato: la reserva genera un <i>hold</i> temporal sobre el inventario, con 15 minutos de vigencia para el pasajero directo.	Decisión con consecuencias fuertes: la operación atómica que debe protegerse contra la sobreventa es la creación del hold , no el pago. El pago posterior solo cambia el estado de la reserva y no vuelve a tocar el inventario.
S-14	Las agencias asociadas operan con tarifa neta más un porcentaje de comisión, y disponen de una ventana de pago de 48 horas con facturación por ciclo, frente a los 15 minutos del pasajero directo.	El escenario menciona agencias pero no su flujo. Lo que realmente cambia es el plazo del hold y el precio aplicado; el resto del proceso reutiliza el mismo motor de reservas e inventario.

3. Requisitos funcionales (5.1)

Prioridad: **Alta** = imprescindible para operar; **Media** = necesario para el negocio completo, no bloqueante; **Baja** = deseable.

La columna *Impl.* indica si el requisito está implementado en código en la primera implementación del backend (sección 8). Su alcance se justifica en la sección 8.5.

ID	Descripción	Actor(es)	Prioridad	Impl.
RF-01	Registrar una cuenta de pasajero con correo y contraseña.	Pasajero	Alta	No

ID	Descripción	Actor(es)	Prioridad	Impl.
RF-02	Autenticar pasajero, agente de agencia, personal de aeropuerto o administrador y emitir un token de sesión con el rol correspondiente.	Pasajero, Agente, Personal, Admin	Alta	No
RF-03	Buscar vuelos por origen, destino, fecha y número de pasajeros, incluyendo itinerarios con escalas (multi-tramo).	Pasajero, Agente	Alta	Sí
RF-04	Mostrar disponibilidad de asientos por clase de cabina para cada vuelo resultante de la búsqueda.	Pasajero, Agente	Alta	Sí
RF-05	Crear una reserva con bloqueo temporal (hold) del inventario de asientos, en estado PENDIENTE_PAGO.	Pasajero, Agente	Alta	Sí
RF-06	Confirmar el pago de una reserva en hold y generar un código de reserva (PNR) único; el pago solo cambia el estado a CONFIRMADA, no vuelve a tocar el inventario.	Pasajero, Agente	Alta	Parcial
RF-07	Expirar automáticamente una reserva en hold no pagada dentro de la ventana permitida (15 min pasajero, 48 h agencia) y liberar el asiento.	Sistema	Alta	No
RF-08	Consultar el estado y detalle de una reserva mediante el código PNR.	Pasajero, Agente	Alta	Sí
RF-09	Modificar el itinerario de una reserva existente, sujeto a disponibilidad, a la ventana de cambio permitida y a la diferencia tarifaria (se cobra si es mayor, no se reembolsa si es menor).	Pasajero, Agente	Alta	No
RF-10	Cancelar una reserva: liberar el asiento sin cargo si aún está en hold, o calcular el reembolso según tarifa y liberar el asiento si ya está confirmada y pagada.	Pasajero, Agente	Alta	No
RF-11	Registrar la afiliación de un pasajero al programa de viajero frecuente.	Pasajero	Media	No
RF-12	Acumular millas automáticamente al completar un vuelo y actualizar el nivel del pasajero.	Sistema	Media	No
RF-13	Redimir millas acumuladas como forma de pago total o parcial de una reserva.	Pasajero, Agente	Media	No
RF-14	Registrar equipaje adicional a una reserva y calcular el cargo correspondiente.	Pasajero, Agente	Media	No
RF-15	Reservar en representación de un pasajero aplicando la tarifa neta de la agencia y registrando la comisión generada.	Agente de Agencia	Alta	Parcial
RF-16	Consultar el reporte de comisiones acumuladas por la agencia en un periodo.	Agente de Agencia	Media	No
RF-17	Realizar el check-in de un pasajero en el mostrador del aeropuerto y emitir la tarjeta de embarque.	Personal de Aeropuerto	Alta	No

ID	Descripción	Actor(es)	Prioridad	Impl.
RF-18	Registrar el embarque de un pasajero en la puerta antes del cierre del vuelo.	Personal de Aeropuerto	Media	No
RF-19	Crear y administrar vuelos recurrentes (plantillas de horario) que generan instancias de vuelo por fecha.	Administrador	Alta	No
RF-20	Administrar tarifas por vuelo y clase de cabina, incluida la tarifa neta de agencias.	Administrador	Media	No
RF-21	Enviar notificaciones automáticas ante confirmación, cambio, cancelación o vencimiento de hold de una reserva.	Sistema	Media	No
RF-22	Registrar en un log de auditoría cada operación de creación, cambio, cancelación o expiración sobre una reserva.	Sistema	Alta	No
RF-23	Generar reportes operativos (ocupación por vuelo, ingresos por periodo, comisiones por agencia) para el administrador.	Administrador	Baja	Sí (OLAP)
RF-24	Permitir check-in en línea (autoservicio) de un pasajero dentro de una ventana previa al vuelo.	Pasajero	Baja	No

Resolución explícita de las preguntas guía

¿Quiénes son los actores y comparten sistema?

Cuatro actores: Pasajero, Agente de agencia asociada, Personal de aeropuerto y Administrador. Todos operan sobre **el mismo backend**, con interfaces distintas por rol mediante RBAC. La razón es de negocio, no de comodidad: si cada canal tuviera su propio backend, no habría forma de garantizar la ausencia de sobreventa *entre* canales, que es precisamente el riesgo que el escenario señala.

¿Cómo se maneja un itinerario multi-tramo?

Una sola reserva con N tramos ordenados (S-02). Se paga, se cambia y se cancela como unidad. Cada tramo hereda su propio estado operativo del vuelo real correspondiente, pero el estado comercial de la reserva es único.

¿Se puede reservar sin pagar?

Sí (S-13). Al reservar se decrementa el inventario de inmediato y la reserva queda en PENDIENTE_PAGO; si no se paga dentro de la ventana, expira y el asiento se libera. La consecuencia de diseño es central: la operación que debe ser atómica frente a la concurrencia es la creación del hold.

¿Qué pasa con cancelaciones y cambios de fecha?

Resuelto en S-04 y S-05. La política de reembolso depende del tipo de tarifa; la liberación del inventario es inmediata en todos los casos.

¿Las agencias tienen un flujo distinto?

Flujo distinto sobre el mismo motor (S-14): cambian el precio aplicado (tarifa neta más comisión) y la ventana de pago (48 horas con facturación por ciclo), no el proceso de reserva ni el control de inventario.

4. Requisitos no funcionales (5.2)

Cada categoría tiene una métrica verificable y un método de verificación declarado. Se indica además el riesgo de negocio que la origina, porque una métrica sin riesgo asociado no permite decidir nada.

ID	Categoría	Requisito con métrica verificable	Método de verificación
RNF-01	Rendimiento	Búsqueda de vuelos: $p95 \leq 300$ ms con 50 solicitudes concurrentes. Creación de reserva: $p95 \leq 800$ ms extremo a extremo.	Prueba de carga con k6 o Locust contra los endpoints de FastAPI.
RNF-02	Concurrencia	Ante ≥ 20 solicitudes simultáneas de creación de hold sobre el último asiento disponible de una instancia de vuelo y clase, exactamente una obtiene el hold y el resto recibe HTTP 409: 0% de sobreventa .	Script con solicitudes POST <code>/reservas</code> concurrentes (asyncio o threading) y verificación en base de datos de que los asientos vendidos no superan el inventario.
RNF-03	Disponibilidad	El endpoint <code>/health</code> responde 200 OK en $\geq 99\%$ de los chequeos (cada 60 s) durante una sesión activa de 4 h. Tiempo de recuperación (RTO) ante caída del servicio ≤ 5 min.	Script de sondeo periódico con registro de marcas de tiempo de caída y recuperación.
RNF-04	Seguridad	100% de contraseñas con hash bcrypt (coste ≥ 12). JWT con expiración ≤ 30 min. 100% de endpoints con validación de esquema mediante Pydantic y cero consultas SQL concatenadas manualmente.	Revisión de código con lista de verificación y prueba de inyección SQL básica sobre los tres endpoints principales, esperando cero vulnerabilidades explotables.
RNF-05	Escalabilidad	Backend sin estado: añadir una segunda instancia de FastAPI tras el <i>reverse proxy</i> requiere cero cambios de código . El esquema soporta un incremento de diez veces en el volumen de reservas sin rediseño de tablas.	Revisión arquitectónica y despliegue de prueba de una segunda instancia en el mismo entorno.
RNF-06	Auditabilidad	100% de las operaciones de creación, cambio o cancelación quedan registradas en la tabla de auditoría (marca de tiempo, actor, acción, entidad, valor anterior y nuevo). Cada transacción es trazable mediante un identificador de correlación presente tanto en los registros de la aplicación como en la auditoría.	Inspección de <code>audit_log</code> tras un flujo de prueba (crear, modificar, cancelar): deben aparecer tres registros correlacionados.

5. Modelo entidad-relación (5.3)

5.1. Diagrama

El modelo se compone de 16 entidades organizadas en cuatro grupos: actores, red de vuelos, reservas y auditoría. La notación empleada es *Crow's Foot*, consistente en todo el diagrama.

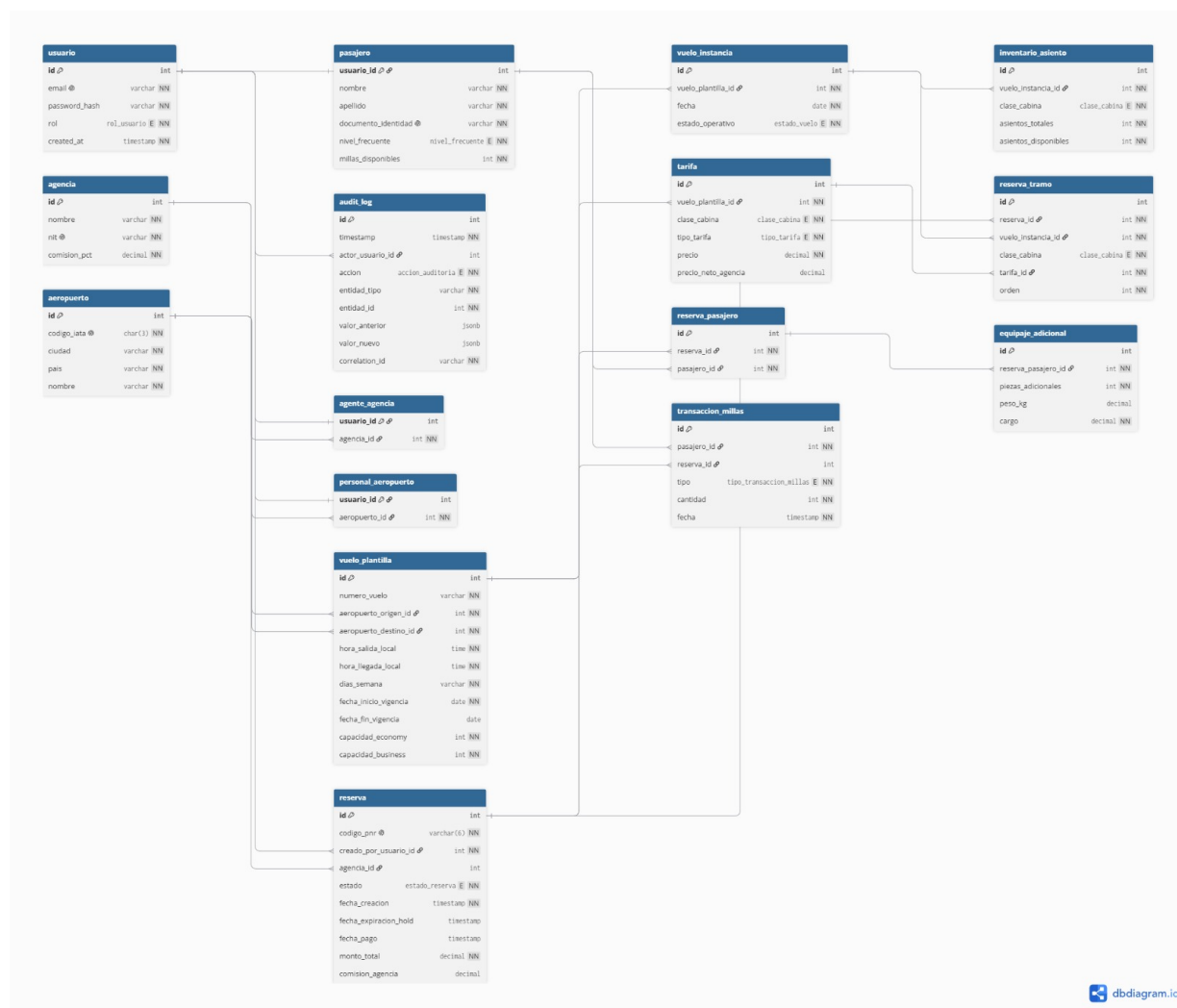


Figura 1: Modelo entidad-relación del sistema de reservas, notación Crow's Foot, 16 entidades agrupadas en actores, red de vuelos, reservas y auditoría.

5.2. Decisiones de modelado y su justificación

Itinerarios de varios tramos. Una reserva no apunta directamente a los vuelos. Entre **reserva** y **vuelo_instancia** se interpone la entidad **reserva_tramo**, que además del vínculo guarda el atributo **orden**, es decir, la posición del tramo dentro del itinerario. Así, un viaje BOG–MDE–CTG es una sola reserva con dos tramos ordenados, y cada tramo puede tener su propia clase de cabina y su propia tarifa. Esto materializa el supuesto S-02.

Vuelos recurrentes. Se separan dos conceptos que suelen confundirse en un solo “vuelo”: **vuelo_plantilla** define la ruta, el número de vuelo, el horario local, los días de la semana en que opera y su periodo de vigencia; **vuelo_instancia** representa la ocurrencia concreta de esa plantilla en una fecha, con su propio estado operativo. La restricción de unicidad sobre (**vuelo_plantilla_id**, **fecha**) impide generar dos veces el mismo vuelo del mismo día. Esto materializa S-07.

Control del inventario. El inventario vive en **inventario_asiento**, una fila por cada par (instancia de vuelo, clase de cabina), con **asientos_totales** y **asientos_disponibles**, y unicidad sobre esa pareja. Es la pieza que hace posible el mecanismo de concurrencia descrito en la sección 6: al concentrar la disponibilidad en una sola fila, el motor de base de datos puede serializar el

acceso sin necesidad de bloqueos explícitos ni infraestructura adicional. Esto materializa S-08.

Ausencia de una entidad *Pago*. El enunciado sugiere *Payment* entre las entidades candidatas y el modelo no la incluye. Es una decisión deliberada derivada de S-11: como el cobro se delega por completo a una pasarela externa, lo único que el sistema necesita registrar es *cuándo* se pagó y *cuánto*, y ambos datos son atributos de la reserva (*fecha_pago*, *monto_total*). Crear una entidad separada para almacenar dos atributos añadiría una relación uno a uno sin ganancia, y tentaría a guardar allí datos de medio de pago que el sistema explícitamente no debe almacenar (RNF-04).

Asignación de asiento fuera de la reserva. Consecuencia de S-08 y S-09: no existe una entidad *Asiento* con filas individuales. Si en el futuro se requiere selección de silla, se añadiría una entidad de asignación vinculada a *reserva_pasajero* y *vuelo_instancia*, sin alterar el mecanismo de control de sobreventa, que seguiría operando sobre el contador.

Fidelización como atributo y como transacción. El nivel de viajero frecuente es un atributo enumerado de *pasajero* (estado actual, consultado en cada reserva) mientras que las millas se modelan como *transaccion_millas*, un histórico de acumulaciones y redenciones. Modelar el nivel como entidad propia añadiría una relación sin información adicional; modelar las millas como un simple saldo impediría auditar cómo se llegó a él.

Nivel de normalización. El modelo transaccional está normalizado hasta 3FN: sin datos derivados almacenados, sin repetición de atributos de ruta en cada instancia de vuelo, y con los enumerados como tipos del motor. Esa normalización es la correcta para el lado OLTP, donde el objetivo es la integridad ante escrituras concurrentes. Sin embargo, es exactamente lo contrario de lo que conviene para responder las preguntas comerciales del escenario: calcular la ocupación por ruta exige unir siete tablas. Por eso el modelo transaccional **no** se replica tal cual en el lado analítico; la sección 9 explica la desnormalización aplicada.

5.3. Diccionario de datos

Grupo 1 — Actores

Atributo	Tipo	Descripción
usuario — cuenta de acceso al sistema, común a todos los roles		
id	int, PK	Identificador único.
email	varchar, único, NN	Credencial de acceso.
password_hash	varchar, NN	Contraseña almacenada como hash (RNF-04).
rol	enum rol_usuario, NN	PASAJERO, AGENTE_AGENCIA, PERSONAL_AEROPUERTO, ADMINISTRADOR.
created_at	timestamp, NN	Fecha de creación de la cuenta.
pasajero — especialización de usuario que viaja		
usuario_id	int, PK, FK → usuario	Relación 1:1 con la cuenta.
nombre	varchar, NN	Nombre del pasajero.
apellido	varchar, NN	Apellido del pasajero.
documento_identidad	varchar, único, NN	Documento con el que viaja.
nivel_frecuente	enum nivel_frecuente, NN	BASICO, PLATA, ORO, PLATINO (S-06).
millas_disponibles	int, NN	Saldo vigente de millas.
agencia — agencia de viajes asociada		

<i>(continuación)</i>		
Atributo	Tipo	Descripción
id	int, PK	Identificador único.
nombre	varchar, NN	Razón social.
nit	varchar, único, NN	Identificación tributaria.
comision_pct	decimal, NN	Porcentaje de comisión pactado (S-14).
agente_agencia — especialización de usuario que reserva por cuenta de una agencia		
usuario_id	int, PK, FK → usuario	Relación 1:1 con la cuenta.
agencia_id	int, FK → agencia, NN	Agencia a la que pertenece.
personal_aeropuerto — especialización de usuario asignado a un aeropuerto		
usuario_id	int, PK, FK → usuario	Relación 1:1 con la cuenta.
aeropuerto_id	int, FK → aeropuerto, NN	Aeropuerto de adscripción.

Grupo 2 — Red de vuelos

Atributo	Tipo	Descripción
aeropuerto		
id	int, PK	Identificador único.
codigo_iata	char(3), único, NN	Código IATA (BOG, MDE, CTG).
ciudad	varchar, NN	Ciudad.
pais	varchar, NN	País.
nombre	varchar, NN	Nombre del aeropuerto.
vuelo_plantilla — definición del vuelo recurrente (S-07)		
id	int, PK	Identificador único.
numero_vuelo	varchar, NN	Número comercial del vuelo.
aeropuerto_origen_id	int, FK → aeropuerto, NN	Origen de la ruta.
aeropuerto_destino_id	int, FK → aeropuerto, NN	Destino de la ruta.
hora_salida_local	time, NN	Hora programada de salida.
hora_llegada_local	time, NN	Hora programada de llegada.
dias_semana	varchar, NN	Días en que opera (un carácter por día).
fecha_inicio_vigencia	date, NN	Inicio del periodo de operación.
fecha_fin_vigencia	date	Fin del periodo; nulo si es indefinido.
capacidad_economy	int, NN	Asientos de económica del equipo asignado.
capacidad_business	int, NN	Asientos de ejecutiva del equipo asignado.
vuelo_instancia — ocurrencia del vuelo en una fecha concreta		
id	int, PK	Identificador único.
vuelo_plantilla_id	int, FK → vuelo_plantilla, NN	Plantilla de la que proviene.
fecha	date, NN	Fecha de operación. Único junto con la plantilla.
estado_operativo	enum estado_vuelo, NN	PROGRAMADO, A_TIEMPO, RETRASADO, CANCELADO, COMPLETADO.
inventario_asiento — control de disponibilidad (S-08)		
id	int, PK	Identificador único.
vuelo_instancia_id	int, FK → vuelo_instancia, NN	Vuelo concreto.

<i>(continuación)</i>		
Atributo	Tipo	Descripción
clase_cabina	enum clase_cabina, NN	ECONOMY o BUSINESS. Único junto con la instancia.
asientos_totales	int, NN	Capacidad de la cabina en ese vuelo.
asientos_disponibles	int, NN	Asientos aún disponibles. Campo sobre el que actúa el mecanismo anti-sobreventa.
tarifa — precio aplicable por clase y tipo		
id	int, PK	Identificador único.
vuelo_plantilla_id	int, FK → vuelo_plantilla, NN	Vuelo al que aplica.
clase_cabina	enum clase_cabina, NN	Cabina a la que aplica.
tipo_tarifa	enum tipo_tarifa, NN	BASICA (no reembolsable) o FLEX (reembolsable).
precio	decimal, NN	Precio de venta al público.
precio_neto_agencia	decimal	Precio neto para agencias; nulo si no aplica (S-14).

Grupo 3 — Reservas

Atributo	Tipo	Descripción
reserva — unidad comercial del viaje (un PNR)		
id	int, PK	Identificador único.
codigo_pnr	varchar(6), único, NN	Código de reserva visible al pasajero.
creado_por_usuario_id	int, FK → usuario, NN	Quién creó la reserva (pasajero o agente).
agencia_id	int, FK → agencia	Agencia asociada; nulo en venta directa.
estado	enum estado_reserva, NN	PENDIENTE_PAGO, CONFIRMADA, CANCELADA, EXPIRADA.
fecha_creacion	timestamp, NN	Momento de creación del hold.
fecha_expiracion_hold	timestamp	Límite para pagar antes de liberar el inventario (S-13, S-14).
fecha_pago	timestamp	Momento del pago; nulo mientras no se pague (S-11).
monto_total	decimal, NN	Importe total de la reserva.
comision_agencia	decimal	Comisión liquidable a la agencia; nulo en venta directa.
reserva_pasajero — pasajeros incluidos en la reserva		
id	int, PK	Identificador único.
reserva_id	int, FK → reserva, NN	Reserva a la que pertenece.
pasajero_id	int, FK → pasajero, NN	Pasajero incluido. Único junto con la reserva.
reserva_tramo — tramos del itinerario (S-02)		
id	int, PK	Identificador único.
reserva_id	int, FK → reserva, NN	Reserva a la que pertenece.
vuelo_instancia_id	int, FK → vuelo_instancia, NN	Vuelo concreto del tramo.
clase_cabina	enum clase_cabina, NN	Cabina contratada en ese tramo.
tarifa_id	int, FK → tarifa, NN	Tarifa aplicada al tramo.
orden	int, NN	Posición del tramo dentro del itinerario.
equipaje_adicional — piezas extra por pasajero (S-10)		
id	int, PK	Identificador único.

<i>(continuación)</i>		
Atributo	Tipo	Descripción
reserva_pasajero_id	int, FK → reserva_pasajero, NN	Pasajero concreto dentro de la reserva.
piezas_adicionales	int, NN	Número de piezas extra.
peso_kg	decimal	Peso declarado.
cargo	decimal, NN	Importe cobrado.
transaccion_millas — histórico de fidelización (S-06)		
id	int, PK	Identificador único.
pasajero_id	int, FK → pasajero, NN	Titular de las millas.
reserva_id	int, FK → reserva	Reserva que origina el movimiento; nulo en ajustes.
tipo	enum tipo_transaccion_millas, NN	ACUMULACION o REDENCION.
cantidad	int, NN	Millas del movimiento.
fecha	timestamp, NN	Momento del movimiento.

Grupo 4 — Auditoría

Atributo	Tipo	Descripción
audit_log — bitácora de cambios (RNF-06, RF-22)		
id	int, PK	Identificador único.
timestamp	timestamp, NN	Momento del evento.
actor_usuario_id	int, FK → usuario	Quién ejecutó la acción; nulo si fue el sistema.
accion	enum accion_auditoria, NN	CREAR, MODIFICAR, CANCELAR, EXPIRAR.
entidad_tipo	varchar, NN	Entidad afectada.
entidad_id	int, NN	Identificador de la fila afectada.
valor_anterior	jsonb	Estado previo, en formato JSON.
valor_nuevo	jsonb	Estado posterior, en formato JSON.
correlation_id	varchar, NN	Identificador que agrupa los eventos de una misma operación.

6. Arquitectura del backend (5.4)

ID	Decisión	Alternativa des- cartada	Justificación (RF/RNF que la sus- tenta)
B1	Monolito modular en FastAPI, desplegado sobre EC2 detrás de Apache como <i>reverse proxy</i> . Módulos previstos por dominio: reservas, inventario, tarifas, agencias, fidelización y auditoría.	Microservicios in- dependientes por dominio.	RNF-05: la métrica exige que añadir una instancia no cueste cambios de código. Un monolito sin estado, replicado detrás del proxy, lo cumple directamente. Microservicios añadirían orquestación, latencia de red entre servicios y complejidad operativa sin ninguna ganancia frente a los requisitos definidos, y fragmentarían la transacción de reserva justo donde se necesita atomicidad (B2).
B2	UPDATE atómico condicional sobre <code>inventario_asiento</code> : se decrementa el contador solo si quedan asientos disponibles, y se decide por el número de filas afectadas (cero filas $\Rightarrow$ HTTP 409).	Bloqueo pesimista (SELECT FOR UPDATE), bloqueo distribuido con Redis, o aislamiento SERIALIZABLE con reintentos.	RNF-02: 0 % de sobreventa sin infraestructura adicional. El motor serializa el acceso a la fila; la garantía no depende del código de aplicación ni del orden de llegada. Además ayuda a RNF-01 , al no mantener transacciones abiertas esperando bloqueos explícitos. Redis quedaba descartado por introducir un componente extra que el presupuesto del entorno no justifica.
B3	Pago delegado íntegramente a una pasarela externa; el sistema registra únicamente momento e importe.	Procesar el cobro dentro del sistema.	RNF-04 y S-11: elimina el almacenamiento de datos de tarjeta y reduce el alcance de cumplimiento. Es también la razón de que el modelo no tenga entidad <i>Pago</i> .
B4	Las agencias consumen la misma API que el canal directo, con identidad y rol propios; lo que cambia es la tarifa aplicada y la ventana de <i>hold</i> .	API B2B separada.	S-14 y RNF-02: un backend único es lo que garantiza que no se venda dos veces el mismo asiento entre canales. Una API separada duplicaría la lógica de inventario, que es exactamente el punto que no debe duplicarse.
B5	El sistema transaccional no atiende consultas analíticas: los reportes se sirven desde una base de datos independiente alimentada por un proceso ETL.	Ejecutar los reportes contra la base transaccional.	RNF-01 y RF-23: una agregación sobre siete tablas compite por CPU y E/S con las escrituras de reserva y degrada el p95 comprometido. El detalle se desarrolla en la sección 9.

7. Arquitectura del frontend (5.5)

Esta sección documenta decisiones de diseño. Conforme al alcance del enunciado, la implementación exigida en esta entrega es la del backend (5.6); el frontend no se implementa.

ID	Decisión	Alternativa des- cartada	Justificación (RF/RNF que la sus- tenta)
F1	Aplicación de página única (SPA) en React, servida como contenido estático por el mismo Apache, sin proceso de servidor adicional.	Renderizado en servidor (SSR).	RNF-01: el sistema es autenticado, de modo que no hay indexación que ganar con SSR. Servir el frontend como estático deja todo el cómputo del servidor disponible para la API, en lugar de competir por CPU con el mismo proceso que atiende las reservas.
F2	Notificación de disponibilidad mediante <i>polling</i> ligero (del orden de 10 s) sobre el vuelo consultado, más una revalidación en el momento de crear el hold.	WebSockets o Server-Sent Events.	RNF-01: una consulta periódica sobre una fila indexada tiene un costo marginal, mientras que mantener conexiones persistentes por usuario añade consumo de memoria y complejidad de despliegue. Punto importante: esto es únicamente experiencia de usuario. La garantía de no sobreventa vive en B2, del lado del servidor; el frontend nunca decide si hay asiento.
F3	Interfaz diferenciada por rol sobre el mismo backend (RBAC), incluyendo una vista específica para agentes de agencia con tarifa neta y comisión visibles.	Aplicación independiente para agencias.	S-14 y B4: el flujo de negocio difiere en precio y plazos, no en el proceso. Separar la interfaz por rol es suficiente y evita duplicar lógica.
F4	Internacionalización y moneda resueltas por catálogo externo de cadenas y formateo en el cliente, con los importes almacenados siempre en una moneda base.	Almacenar importes ya convertidos por moneda.	Guardar importes convertidos obligaría a reescribir históricos ante cada cambio de tasa y haría irreproducibles los reportes de ingresos (RF-23).

8. Primera implementación del backend (5.6)

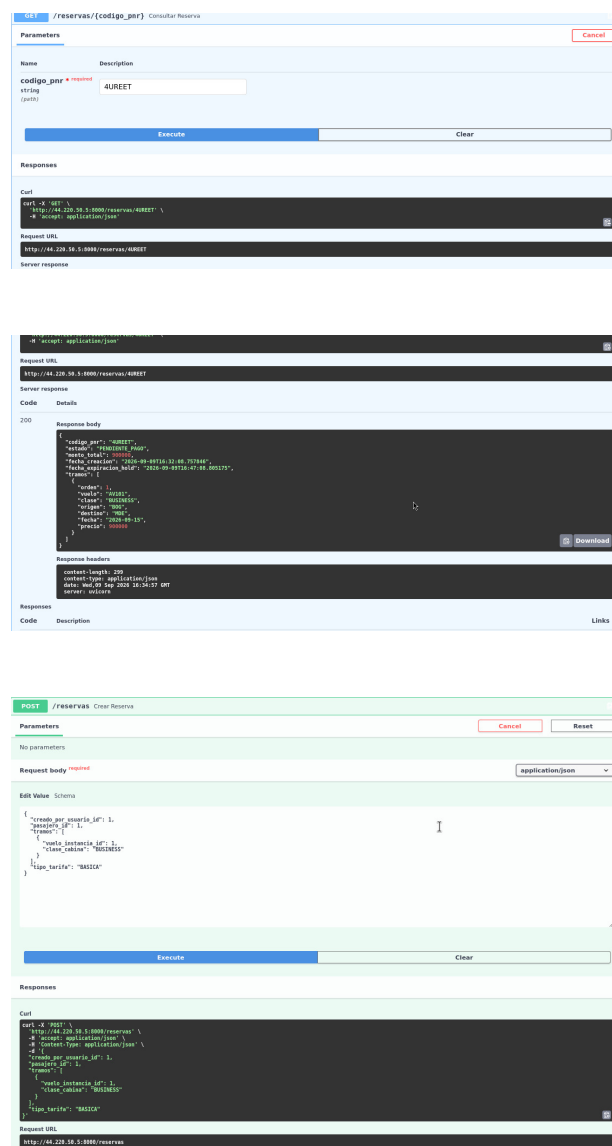
8.1. Alcance construido

El enunciado fija como alcance mínimo una base de datos funcional, una API en FastAPI que exponga la búsqueda de vuelos, la creación de reserva y la consulta de reserva, y que la creación refleje el mecanismo de concurrencia decidido en 5.4. Eso es exactamente lo implementado:

- **Base de datos:** las 16 entidades del modelo, con sus tipos enumerados, claves foráneas, restricciones de unicidad e índices, creadas mediante un script DDL sobre una base PostgreSQL en Amazon RDS.
- **API:** GET /vuelos (búsqueda con disponibilidad y tarifa), POST /reservas (creación del hold), GET /reservas/{codigo_pnr} (consulta) y GET /health (verificación de estado, usada por RNF-03).
- **Prueba de concurrencia:** script automatizado que dispara 20 solicitudes simultáneas contra el último asiento y verifica el estado final en la base de datos.

8.2. Estructura del código

- `app/main.py` — definición de las rutas, cada una anotada con el RF que satisface.
- `app/database.py` — pool de conexiones a PostgreSQL, parametrizado por variables de entorno.
- `app/schemas.py` — modelos Pydantic de entrada y salida.
- `schema.sql` — DDL de las 16 tablas y sus tipos enumerados.
- `seed.py` — datos mínimos para la prueba: una ruta BOG–MDE con una sola plaza en económica y doce en ejecutiva.
- `test_concurrency.py` — prueba automatizada de concurrencia.
- `olap_reporte.sql` — consulta de desnormalización que alimenta el lado analítico (sección 9).



POST /reservas

Parameters

Name	Description
origen	origen
destino	destino
fecha	fecha
pasajeros	Default value: 1

Responses

8.3. Mecanismo de concurrencia implementado

La creación de la reserva no consulta la disponibilidad y decide después. Ambas cosas ocurren en una sola instrucción, cuya condición forma parte del propio UPDATE:

```
UPDATE inventario_asiento
SET asientos_disponibles = asientos_disponibles - 1
WHERE vuelo_instancia_id = $1
AND clase_cabina = $2
AND asientos_disponibles > 0;
```

Listing 1: Decremento condicional del inventario (decisión B2).

Si la instrucción afecta una fila, la solicitud obtuvo el hold y la reserva se crea. Si afecta cero filas, no había disponibilidad y la API responde **409 Conflict**. La diferencia con leer primero y restar después es determinante: entre la lectura y la escritura otra solicitud podría colarse y ambas crearían tener el último asiento. Al condicionar el propio UPDATE, el motor de base de datos serializa el acceso a la fila y la garantía deja de depender del código de aplicación.

8.4. Evidencia de la prueba de concurrencia

El enunciado exige evidencia concreta de que el mecanismo previene efectivamente la sobreventa. La prueba se ejecutó contra la instancia real de RDS y el servidor desplegado en EC2, no contra un entorno local.

Diseño de la prueba. Se dejó exactamente una plaza disponible en la cabina económica del vuelo de prueba y se dispararon 20 solicitudes **POST /reservas** simultáneas contra esa misma plaza. Las solicitudes se lanzan con `asyncio.gather()`, que ejecuta las 20 corrutinas sin esperar a que termine la anterior; no es un bucle secuencial. El criterio de aprobación es triple: exactamente una respuesta 201, exactamente diecinueve respuestas 409, y contador final igual a cero, nunca negativo.

Resultado obtenido.

Indicador	Resultado
Solicitudes simultáneas lanzadas	20
Respuestas 201 Created (hold obtenido)	1
Respuestas 409 Conflict (sin disponibilidad)	19
PNR de la reserva ganadora	4XRGQ8
asientos_disponibles en ECONOMY antes	1
asientos_disponibles en ECONOMY después	0
asientos_disponibles en BUSINESS antes y después	12 / 12
Sobreventa registrada	0

La cabina BUSINESS funciona como control: permanece intacta en 12, lo que demuestra que la operación afectó únicamente la fila en disputa y no el inventario en general.

```
python test_concurrency.py # contra localhost:8000
python test_concurrency.py http://EC2_IP:8000 # contra el EC2 del lab

'''
import asyncio
import os
import sys
from collections import Counter

import asyncpg
import httpx

BASE_URL = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:8000"
N_SOLICITUDES = 20 # mismo N que la RNF de Concurrency del documento
VUELO_INSTANCIA_ID = 1
CLASE = "ECONOMY"

DBN = {
    "postgresql": (os.getenv("DB_USER", "postgres"), (os.getenv("DB_PASSWORD", "postgres"))),
    "postgres": (os.getenv("DB_HOST", "localhost"), (os.getenv("DB_PORT", "5432"))),
    "": (os.getenv("DB_NAME", "testdb"))
}

@pytest.fixture
def db_client():
    """
    (venv) i-//fast-backend $ psql "host=$DB_HOST port=$DB_PORT dbname=$DB_NAME user=$DB_USER password=$DB_PASSWORD sslmode=require" \
    -c "UPDATE inventario_asiento SET asientos_disponibles = 1 WHERE vuelo_instancia_id = 1 AND clase_cabina = '$CONOMY';"
    UPDATE 1
    (venv) i-//fast-backend $ psql "host=$DB_HOST port=$DB_PORT dbname=$DB_NAME user=$DB_USER password=$DB_PASSWORD sslmode=require" \
    -c "SELECT vuelo_instancia_id, clase_cabina, asientos_disponibles, ROW() FROM inventario_asiento WHERE vuelo_instancia_id = 1;"
    vuelo_instancia_id | clase_cabina | asientos_disponibles |
    -----
    1 | ECONOMY | 1 | 2026-09-08 20:17:14.626109+00
    1 | BUSINESS | 12 | 2026-09-08 20:17:14.626109+00
    (2 rows)
    """
```

Figura 2: Estado del inventario inmediatamente antes de la prueba: una plaza disponible en ECONOMY y doce en BUSINESS, con marca de tiempo 20:17:14 UTC.

```

aws Cloud9
[Alt+S]

httpx.ConnectError: All connection attempts failed
(venv) :~/fase4-backend $ curl http://localhost:8000/health
curl: (7) Failed to connect to localhost port 8000 after 0 ms: Connection refused
(venv) :~/fase4-backend $ uvicorn app.main:app --host 0.0.0.0 --port 8000 &
[1] 1880
(venv) :~/fase4-backend $ INFO:      Started server process [1880]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

(venv) :~/fase4-backend $ curl http://localhost:8000/health
INFO: 127.0.0.1:39042 - "GET /health HTTP/1.1" 200 OK
{"status":"ok"} (venv) :~/fpython test_concurrency.py http://100.55.95.232:8000 100.55.95.232:8000
INFO: 100.55.95.232:43838 - "POST /reservas HTTP/1.1" 201 Created
INFO: 100.55.95.232:43842 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43900 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43874 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43858 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43908 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43958 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43850 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43846 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43886 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43924 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43822 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43862 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43966 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43926 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43942 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43938 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43978 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43944 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43964 - "POST /reservas HTTP/1.1" 409 Conflict
Disparadas 20 solicitudes simultáneas contra vuelo instancia id=1, clase=ECONOMY (1 asiento disponible)

```

Figura 3: Veinte solicitudes simultáneas contra la última plaza: una respuesta 201 **Created** y diecinueve 409 **Conflict**. En la parte superior se observa un intento previo fallido (**ConnectError** y **Connection refused**) causado por el reinicio de la sesión del entorno, que dejó el servidor detenido; ese intento nunca alcanzó la API, razón por la cual el inventario se mantuvo en una plaza. Se relanzó el servidor y se repitió la prueba.

8.5. Desviaciones y limitaciones declaradas del prototipo

El enunciado pide explícitamente declarar cualquier desviación respecto al diseño de 5.3 y 5.4. Declararla no es un demérito: es información de valor.

```

INFO: 100.55.95.232:43822 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43862 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43966 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43926 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43942 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43938 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43978 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43944 - "POST /reservas HTTP/1.1" 409 Conflict
INFO: 100.55.95.232:43964 - "POST /reservas HTTP/1.1" 409 Conflict
Disparadas 20 solicitudes simultáneas contra vuelo_instancia_id=1, clase=ECONOMY (1 asiento disponible)

Códigos de respuesta: {409: 19, 201: 1}
  201 Created (obtuvieron el hold): 1
  409 Conflict (sin disponibilidad): 19

Reserva creada: PNR=4XRGQ8 (solicitud #1)

asientos_disponibles en BD tras la prueba: 0

✓ PASÓ: exactamente 1 hold, 19 rechazos, 0 asientos sobrantes, sin sobreventa.
(venv) ~/fase4-backend $ psql "host=$DB_HOST port=$DB_PORT dbname=$DB_NAME user=$DB_USER password=$DB_PASSWORD sslmode=require" \
-c "SELECT vuelo_instancia_id, clase_cabina, asientos_disponibles, NOW() FROM inventario_asiento WHERE vuelo_instancia_id = 1;"
vuelo_instancia_id | clase_cabina | asientos_disponibles | now
-----
1 | ECONOMY      | 0 | 2026-09-08 20:23:39.332385+00
1 | BUSINESS    | 12 | 2026-09-08 20:23:39.332385+00
(2 rows)

(venv) ~/fase4-backend $
Broadcast message from root@ip-172-31-74-122 (Tue 2026-09-08 20:25:02 UTC):

The system is going down for poweroff at Tue 2026-09-08 20:35:02 UTC!

```

Figura 4: Resumen impreso por el script de prueba ({409: 19, 201: 1}, PNR 4XRGQ8, contador en cero) y verificación posterior directa en la base de datos: ECONOMY en 0 y BUSINESS intacta en 12, con marca de tiempo 20:23:39 UTC.

D1. Modularidad por capa técnica, no por dominio. La decisión B1 plantea un monolito modular con seis módulos por dominio. Lo implementado está organizado por capa técnica (rutas, acceso a datos, validación), no por dominio de negocio. La condición de monolito se cumple sin reservas — un solo proceso, un solo despliegue, RNF-05 satisfecho — pero la separación en módulos de dominio sigue siendo diseño, no código. El motivo es de alcance: con tres endpoints, seis paquetes serían estructura vacía. La reorganización es directa cuando el número de operaciones lo justifique.

D2. Alcance de la API frente al catálogo de 24 RF. El mínimo exigido por 5.6 es “searching available flights, creating a reservation, and querying an existing reservation”. Las tres operaciones están implementadas y probadas, y corresponden a RF-03 y RF-04 (búsqueda con disponibilidad por cabina), RF-05 (creación del hold) y RF-08 (consulta por PNR). RF-15 y RF-06 quedan parcialmente cubiertos, en el sentido explicado en D3 y D6. Los requisitos restantes están modelados en la base de datos y decididos en la arquitectura, pero fuera del alcance de este prototipo: el objetivo declarado de esta fase es validar que las decisiones de diseño son implementables y que el mecanismo de concurrencia funciona, no construir el sistema completo.

D3. Cobertura real del modelo de datos por el código. Distinguir esto es importante para la sustentación:

Aspecto	Estado en el prototipo	Diseño completo (RNF-04)
Autenticación	Los endpoints no verifican token	JWT con rol, RBAC por endpoint
Contraseñas	Valor de demostración en los datos sembrados	Hash con bcrypt en <code>password_hash</code>
Transporte	HTTP en la instancia del laboratorio	TLS terminado en el proxy
Datos de tarjeta	No se almacenan (cumple ya)	No se almacenan (pasarela externa)
Acceso a la base	Restringido por Security Group (cumple ya)	Restringido por Security Group

Diez de las dieciséis entidades tienen datos reales generados por el código; seis existen únicamente como esquema.

D4. Frontend no implementado. La sección 5.5 del enunciado pide decisiones de arquitectura de frontend, no código; la implementación exigida en esta entrega es la del backend (5.6). Las decisiones F1 a F4 están tomadas y justificadas contra RNF-01.

D5. Simplificaciones de seguridad del prototipo.

Aspecto	Estado en el prototipo	Diseño completo (RNF-04)
Autenticación	Los endpoints no verifican token	JWT con rol, RBAC por endpoint
Contraseñas	Valor de demostración en los datos sembrados	Hash con bcrypt en <code>password_hash</code>
Transporte	HTTP en la instancia del laboratorio	TLS terminado en el proxy
Datos de tarjeta	No se almacenan (cumple ya)	No se almacenan (pasarela externa)
Acceso a la base	Restringido por Security Group (cumple ya)	Restringido por Security Group

Ninguna de estas simplificaciones afecta al mecanismo que sí fue objeto de prueba: la garantía de no sobreventa opera en la capa de base de datos y es independiente de la autenticación.

D6. Momento de generación del PNR. Aunque RF-06 indica que el PNR se genera al confirmar el pago, la implementación lo genera al crear el *hold*. Esto permite que el pasajero pueda consultar su reserva antes de pagar. Por ello, RF-06 se entiende como la confirmación del pago y cambio de estado a CONFIRMADA, mientras que la generación del PNR queda asociada a RF-05.

D7. Detalles menores de implementación. La creación de reserva admite un pasajero por llamada, aunque el modelo (`reserva_pasajero`) soporta varios. El código PNR se genera sin reintento ante colisión. La expiración automática de holds (RF-07) requeriría un proceso programado que no forma parte de este alcance.

9. Implementación en AWS: ETL, catálogo de datos y costos (5.7)

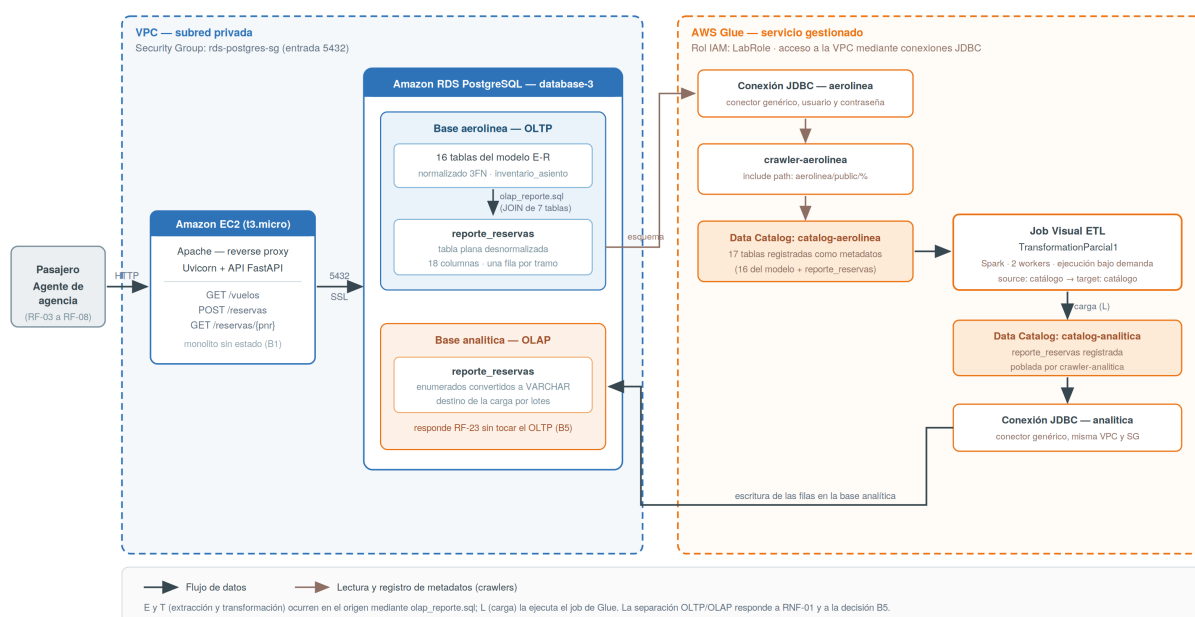
9.1. Arquitectura desplegada

La implementación conecta el sistema transaccional con una base de datos analítica independiente mediante un proceso ETL, y registra ambas bases en el AWS Glue Data Catalog.

- **Instancia RDS PostgreSQL** que aloja dos bases de datos independientes: **aerolinea** (OLTP, las 16 tablas del modelo) y **analitica** (OLAP, destino del ETL).
- **Instancia EC2** que ejecuta la API FastAPI mediante Uvicorn detrás de Apache.
- **Dos conexiones JDBC de Glue**, una por base de datos, dentro de la misma VPC, subred y Security Group que la instancia RDS.
- **Dos crawlers de Glue**, uno por conexión, que registran los metadatos en dos bases lógicas del catálogo: **catalog-aerolinea** y **catalog-analitica**.
- **Un job de Glue Visual ETL** que lee la tabla de reporte desde el catálogo de origen y la escribe en la base analítica.

Arquitectura desplegada: sistema transaccional, proceso ETL y Glue Data Catalog

Sistema de Reservas de Aerolínea — región us-east-1



9.2. Qué se copia al lado analítico y por qué

Las preguntas sobre el negocio, como cuántos asientos se ocupan en cada ruta, cuánto dinero genera cada tarifa y cuántas reservas se cancelan, necesitan información de varias tablas. Por eso, hacer todas esas conexiones cada vez que se genera un reporte podría hacer que el sistema sea más lento.

Para evitarlo, se decidió preparar la información antes y copiarla a una tabla llamada **reporte-reservas**. Esta tabla reúne en un solo lugar los datos importantes de cada tramo reservado, como el estado de la reserva, fechas, precio, agencia, comisión, clase, vuelo, ruta y tipo de tarifa.

Después, AWS Glue copia esta información ya organizada hacia la base de datos donde se realizan los reportes. Así, los reportes pueden generarse más rápido y sin afectar el funcionamiento normal del sistema.

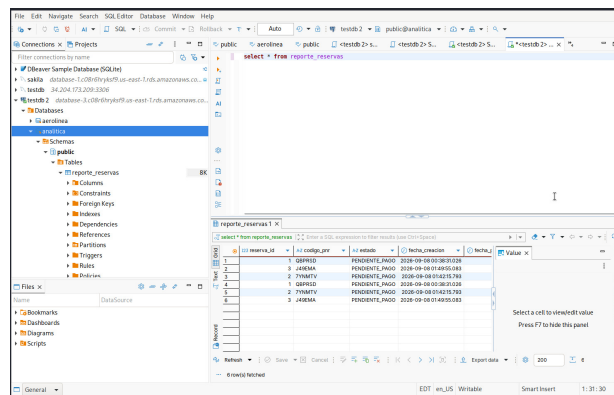
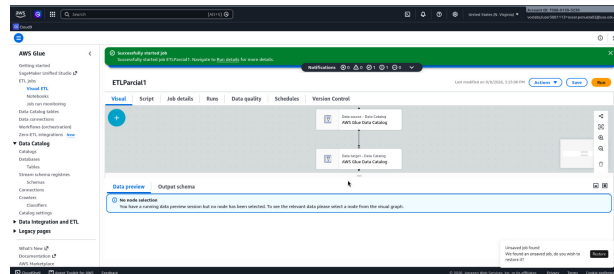
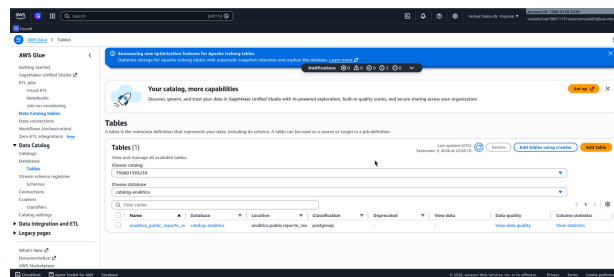
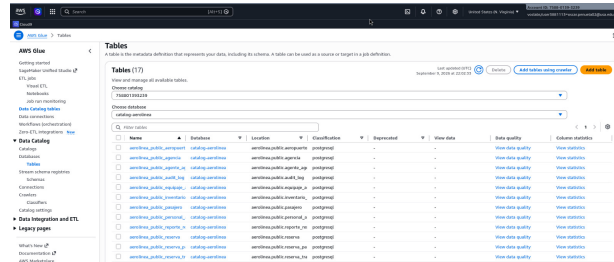
¿Debe el esquema analítico ser igual al transaccional? No. Cada uno tiene un objetivo diferente:

1. **Normalización.** El sistema transaccional usa 3FN para evitar errores y mantener los datos organizados. El sistema analítico usa una tabla plana para hacer los reportes más rápido y evitar muchas uniones.

2. **Tipos de datos.** En el sistema transaccional algunos campos usan valores enumerados para evitar datos incorrectos. En el sistema analítico se usan VARCHAR, porque los datos ya están validados y así es más fácil realizar cambios futuros.

9.3. Registro en el Glue Data Catalog

Ambas bases quedan registradas como metadatos consultables desde Glue, en dos bases lógicas separadas del catálogo.



9.4. Obstáculos técnicos resueltos durante la implementación

Se documentan porque explican por qué la configuración final tiene la forma que tiene, y no otra aparentemente más simple.

Obstáculo	Resolución y motivo
Conector PostgreSQL de Glue falla con STS.	Se cambió al conector JDBC genérico , usando usuario y contraseña directamente.
La subred de una conexión no se puede modificar.	Se eliminó la conexión y se creó nuevamente con la configuración de red correcta.
No se pueden crear nuevos roles IAM.	Se utilizó el rol existente LabRole , que ya tenía los permisos necesarios.
El crawler terminaba sin registrar tablas.	El <i>include path</i> tenía un carácter sobrante. Se corrigió a base/esquema/% y el crawler registró la tabla.
El destino de Visual ETL exigía una tabla del catálogo.	Se creó la tabla vacía en analitica , se ejecutó el crawler y luego se seleccionó la tabla en Glue.
Conexión rechazada por autenticación.	Se agregó sslmode=require para permitir la conexión cifrada con RDS.

9.5. Justificación de los servicios AWS empleados

Cada servicio se justifica contra un requisito propio del documento y contra uno o más pilares del AWS Well-Architected Framework.

Servicio AWS	Alternativa considerada	Requisito que sustenta la elección	Pilar(es) Well-Architected
Amazon RDS for PostgreSQL	PostgreSQL en EC2; Aurora Serverless	RNF-02 y RNF-03 : atomicidad y disponibilidad del 99,9 %.	<i>Fiabilidad; Excelencia operativa</i> : copias, recuperación y mantenimiento gestionados.
Base de datos aerolinea (OLTP)	Una base compartida para operación y análisis	RNF-01 , RNF-02 : separar las operaciones de los análisis.	<i>Eficiencia del rendimiento</i> : cada carga usa el esquema adecuado.
Base de datos analitica (OLAP)	Redshift; consultas directas al OLTP	RF-23 , B5 : generar reportes sin afectar las ventas.	<i>Eficiencia del rendimiento; Optimización de costos</i> .
Amazon EC2 (API FastAPI)	AWS Lambda; Amazon ECS	RNF-05 : permitir réplicas sin cambiar el código.	<i>Fiabilidad; Optimización de costos</i> : adecuada para carga constante.
AWS Glue Visual ETL	AWS DMS; script en EC2	RF-23 : mover los datos de forma reproducible y controlada.	<i>Optimización de costos; Sostenibilidad; Excelencia operativa</i> .
AWS Glue Crawlers	Registro manual de tablas	RNF-06 : mantener actualizado el esquema automáticamente.	<i>Excelencia operativa</i> : descubrimiento automático de tablas.
AWS Glue Data Catalog	Documentación externa	RF-23 : disponer de metadatos para cargas y consultas analíticas.	<i>Excelencia operativa; Eficiencia del rendimiento</i> .
IAM (LabRole) y red (VPC, subred y Security Group)	Base de datos pública con credenciales en el código	RNF-04 : restringir el acceso y proteger las credenciales.	<i>Seguridad; Fiabilidad</i> : aislamiento y menor exposición.

9.6. Proyección de costos

Supuestos declarados.

- Región us-east-1, precios bajo demanda, sin instancias reservadas ni plan de ahorro.
- Mes de 730 horas; ambas instancias operando de forma continua.
- Una única instancia RDS `db.t3.micro` Single-AZ alojando las dos bases de datos, con 20 GB de almacenamiento gp3.
- Una instancia EC2 `t3.micro` con 8 GB de EBS gp3 para la API.
- Job de Glue de 2 workers (2 DPU), ejecución diaria, con facturación mínima por ejecución.
- Crawlers ejecutados semanalmente, 1 DPU, con el mínimo facturable de 10 minutos por ejecución.
- Glue Data Catalog dentro del millón de objetos gratuitos.
- Tráfico entre EC2 y RDS dentro de la misma zona de disponibilidad, sin costo de transferencia.

Concepto	Supuesto de dimensionamiento	USD/mes
Instancia RDS PostgreSQL	db.t3.micro, Single-AZ, 730 h	13,14
Almacenamiento RDS	20 GB gp3	2,30
Instancia EC2 (API)	t3.micro, 730 h	7,59
Almacenamiento EBS	8 GB gp3	0,64
Job de Glue ETL	2 DPU × 30 ejecuciones	2,20
Crawlers de Glue	1 DPU × 8 ejecuciones	0,60
Glue Data Catalog	Dentro de capa gratuita	0,00
Transferencia de datos	Intra-AZ	0,00
Total escenario base		26,47

Escenario de crecimiento por diez. Se modela un aumento simultáneo del volumen de datos y de la frecuencia de actualización: almacenamiento de 20 a 200 GB, instancia de base de datos elevada a `db.t3.small`, una segunda instancia de aplicación para absorber el tráfico, ejecución del ETL diez veces más frecuente (300 ejecuciones mensuales) y crawlers diarios.

Concepto	Dimensionamiento ×10	USD/mes
Instancia RDS PostgreSQL	db.t3.small, Single-AZ	26,28
Almacenamiento RDS	200 GB gp3	23,00
Instancias EC2 (API)	2 × t3.micro	15,18
Almacenamiento EBS	2 × 8 GB gp3	1,28
Job de Glue ETL	2 DPU × 300 ejecuciones	22,00
Crawlers de Glue	1 DPU × 60 ejecuciones	4,40
Glue Data Catalog	Dentro de capa gratuita	0,00
Total escenario ×10		92,14

Lectura del resultado. Al aumentar diez veces el volumen y la frecuencia, el costo aumenta aproximadamente 3,5 veces, no diez. Esto se debe a que instancias y almacenamiento tienen un costo fijo, mientras que los costos de Glue se distribuyen entre más ejecuciones. Esto muestra una mejora en la eficiencia y apoya el pilar de *Optimización de costos*.

Desde la *Sostenibilidad*, usar 2 workers en lugar de 10 evita consumir recursos innecesarios. Si el sistema crece, también se puede ajustar la frecuencia de ejecución según la necesidad de actualización de los datos.

10. Trazabilidad cruzada

Cada decisión de arquitectura e infraestructura puede rastrearse hasta el requisito que la origina y hasta la evidencia que la verifica.

ID	Decisión	Origen	Verificación
B1	Monolito modular sobre EC2	RNF-05	Proceso sin estado; réplica sin cambios de código
B2	UPDATE atómico condicional sobre el inventario	RNF-02, S-08	Prueba de concurrencia, sección 8.4
B3	Pago delegado a pasarela externa	RNF-04, S-11	Ausencia de campos de tarjeta en el esquema
B4	Misma API para agencias y canal directo	S-14, RNF-02	Campo <code>agencia_id</code> en la reserva; inventario único
B5	Separación OLTP/OLAP	RNF-01, RF-23	Pipeline de la sección 9
F1	SPA estática servida por el proxy	RNF-01	Decisión de diseño; no implementada (D4)
F2	Polling ligero de disponibilidad	RNF-01	Decisión de diseño; garantía real en B2
F3	Interfaz diferenciada por rol	S-14, B4	Decisión de diseño; roles presentes en el modelo
F4	Importes en moneda base	RF-23	Modelo: <code>monto_total</code> y <code>precio</code> sin moneda convertida
M1	<code>reserva_tramo</code> con atributo <code>orden</code>	S-02, RF-03	Diagrama E-R y diccionario de datos
M2	<code>vuelo_plantilla</code> frente a <code>vuelo_instancia</code>	S-07	Diagrama E-R; unicidad (plantilla, fecha)
M3	Inventario como contador por cabina	S-08, RNF-02	Tabla <code>inventario_asiento</code> ; prueba de concurrencia
M4	Sin entidad <i>Pago</i>	S-11, RNF-04	Atributos <code>fecha_pago</code> y <code>monto_total</code> en la reserva
M5	Sin asignación de silla en la reserva	S-09	Ausencia de entidad de asiento individual
A1	Tabla analítica desnormalizada	RF-23, B5	<code>reporte_reservas</code> verificada en destino
A2	Enumerados convertidos a texto en OLAP	RF-23	DDL de la tabla de destino
A3	Conector JDBC genérico en lugar del unificado	Restricción del entorno	Conexiones en estado <i>Ready</i>
A4	Job de 2 workers en lugar de 10	Optimización de costos	Configuración del job; sección 9.6
A5	Registro de ambas bases en el catálogo	RNF-06, RF-23	Pantallazos 9 y 10

11. Limitaciones y trabajo futuro

- Los 21 requisitos fuera del prototipo están diseñados, pero no implementados. Se recomienda continuar con RF-06 y RF-07 para completar el ciclo del *hold*.
- `audit_log` está modelado, pero aún no registra cambios ni se incluye en el proceso analítico. Sería el siguiente candidato para el pipeline.

- Las medidas de seguridad de D5 (autenticación, contraseñas y TLS) deben implementarse antes de un despliegue real.
- El frontend está diseñado, pero no construido (D4).
- La restricción específica del equipo aún no ha sido integrada al diseño.